# Inheritance

**Inheritance in Java** is a mechanism in which one object acquires all the properties and behaviors of a parent object.

class that is inherited is called a ***superclass (parent class).***

- *The class inherits the propertieis called* a ***subclass.(child class)***
- *Therefore, a subclass is a specialized version of a superclass. It inherits all of the* instance variables and methods defined by the superclass and adds its own, unique elements.

- Using inheritance, we can create a general class that defines related items. This class can then be inherited by other, more specific classes, each adding those things that are unique to it. In the terminology of Java, a class this inherited is called a *superclass*.
- The class that does the inheriting is called a *subclass*. Therefore, a subclass is a specialized version of a superclass. It inherits all of the members defined by the superclass and adds its own, unique elements.

```
// This program uses inheritance to extend Box.
class Box {
  double width;
  double height;
  double depth;

  // construct clone of an object
  Box(Box ob) { // pass object to constructor
    width = ob.width;
    height = ob.height;
    depth = ob.depth;
  }

  // constructor used when all dimensions specified
  Box(double w, double h, double d) {
    width = w;
    height = h;
    depth = d;
  }
```

```
  // constructor used when cube is created
  Box(double len) {
    width = height = depth = len;
  }

  // compute and return volume
  double volume() {
    return width * height * depth;
  }
}

// Here, Box is extended to include weight.
class BoxWeight extends Box {
```

```
  double weight; // weight of box

  // constructor for BoxWeight
  BoxWeight(double w, double h, double d, double m) {
    width = w;
    height = h;
    depth = d;
    weight = m;
  }
}
```

```
class DemoBoxWeight {
  public static void main(String args[]) {
    BoxWeight mybox1 = new BoxWeight(10, 20, 15, 34.3);
    BoxWeight mybox2 = new BoxWeight(2, 3, 4, 0.076);
    double vol;

    vol = mybox1.volume();
    System.out.println("Volume of mybox1 is " + vol);
    System.out.println("Weight of mybox1 is " + mybox1.weight
    System.out.println();

    vol = mybox2.volume();
    System.out.println("Volume of mybox2 is " + vol);
    System.out.println("Weight of mybox2 is " + mybox2.weight
  }
}
```

```
class Vehicle {
  String brand = "Ford";
  String name;
  Vehicle()
   {
   brand="xyz";
   }
  public void honk() {
    System.out.println("Tuut, tuut!");
  }
}
class Car extends Vehicle {
  private String modelName = "Mustang";
  public static void main(String[] args) {
    Car myFastCar = new Car();
    myFastCar.honk();
    System.out.println(myFastCar.brand + " " + myFastCar.modelName+"
"+myFastCar.name);
```

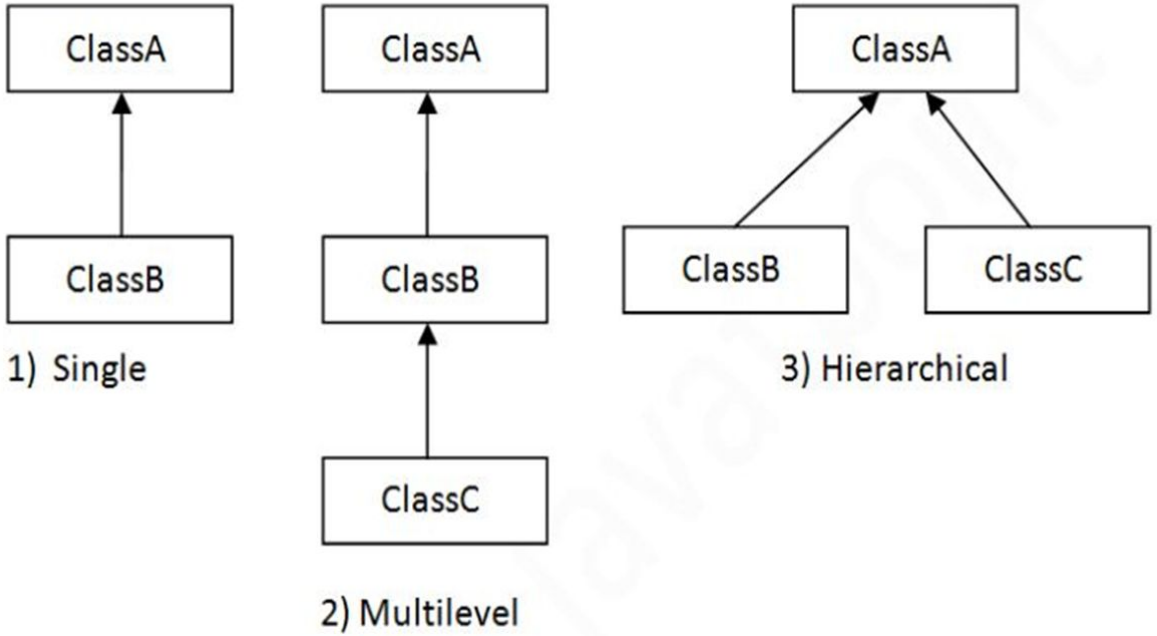
# Member Access and Inheritance

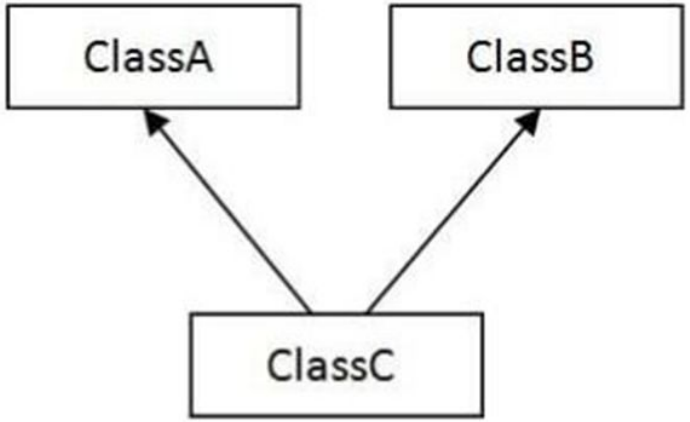
Although a subclass includes all of the members of its superclass, it cannot access those members of the superclass that have been declared as **private.**

```
// Create a superclass.
class A {
int i; // public by default
private int j; // private to A
void setij(int x, int y) {
i = x;
j = y;
}
}
```

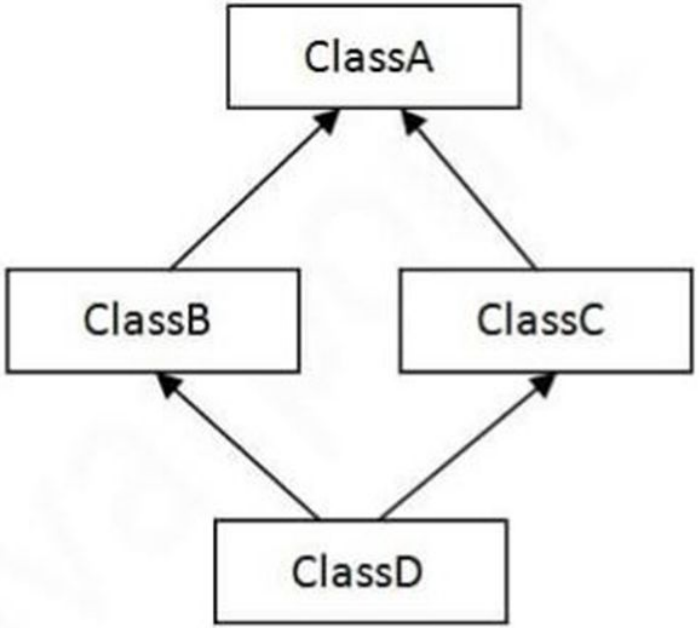
```
// A's j is not accessible here.
class B extends A {
int total;
void sum() {
total = i + j; // ERROR, j is not accessible here
}
}
class Access {
public static void main(String args[]) {
B subOb = new B();
subOb.setij(10, 12);
subOb.sum();
System.out.println("Total is " + subOb.total);
}
}
```

# Types of inheritance in java






4) Multiple




5) Hybrid

Java does not support Multiple inheritance or Hybrid as it might create ambiguity

# Using Super

Whenever a subclass needs to refer to its immediate super class, it can do so by use of the keyword **super.**

**super has two general forms.**

- The first calls the super class' constructor.
- The second is used to access a member of the super class that has been hidden by a member of a subclass.

**Using super to Call Superclass Constructors**

A subclass can call a constructor defined by its super class by use of the following form of super:

***super(arg-list);***

Here, *arg-list specifies any arguments needed by the constructor in the superclass.*

- *super( )* must always be the first statement executed inside a subclass' constructor.
- Java implicitly call the super class constructor if user do not include it as the first statement of constructor

```
class A{
A(int a){  System.out.println(“ BIT);}
}
class D extends A{
D(){
super(a);
System.out.println(“BANGALORE”);
}
}
class TestSuper3{
public static void main(String args[]){
D OB1=new D();
}}
```

# Usage of Java super Keyword

- super can be used to refer immediate parent class instance variable.
- super can be used to invoke immediate parent class method.
- super() can be used to invoke immediate parent class constructor.

**Note: super keyword can only be used in the subclass**

Most applicable to situations in which member names of a subclass hide members by the same name in the superclass.

**super( ) is used, consider this improved version of the BoxWeight( ) class:**

```
// BoxWeight now uses super to initialize its Box attributes.
class BoxWeight extends Box {
double weight; // weight of box
// initialize width, height, and depth using super()
BoxWeight(double w, double h, double d, double m) {
super(w, h, d); // call super class constructor
weight = m;
}
}
```

**super is used to refer immediate parent class instance variable**

```
class A{
String color="white";
}
class D extends  A{
String color="black";
void printColor(){
System.out.println(color);//prints color of D class
System.out.println(super.color);//prints color of A class
}
}
class TestSuper1{
public static void main(String args[]){
D d=new D();
d.printColor();
}}
```

```
// Using super to overcome name hiding.
class A {
int i;
}
// Create a subclass by extending class A.
class B extends A {
int i; // this i hides the i in A
B(int a, int b) {
super.i = a; // i in A
i = b; // i in B
}
void show() {
System.out.println("i in superclass: " + super.i);
System.out.println("i in subclass: " + i);
}
}
```

```
class UseSuper {
public static void main(String args[]) {
B subOb = new B(1, 2);
subOb.show();
}
}
```

This program displays the following:

```
i in superclass: 1
i in subclass: 2
```

Although the instance variable i in B hides the i in A, super allows access to the i defined in the superclass.

As you will see, super can also be used to call methods that are hidden by a subclass.

# Creating a Multilevel Hierarchy

- When there is a chain of inheritance, it is known as *multilevel inheritance*.
- For example, given three classes called A, B, and C, C can be a subclass of B, which is a subclass of A.

- When this type of situation occurs, each subclass inherits all of the traits found in all of its super classes. In this case, C inherits all aspects of B and A.

```
class Shape {
 public void display()
{
  System.out.println("Inside display");
 } }
class Rectangle extends Shape {
 public void area() {
System.out.println("Inside area");
} }
class Cube extends Rectangle {
  public void volume() {
  System.out.println("Inside volume");    } }
public class Tester {
public static void main(String[] arguments) {
Cube cube1 =  new Cube(); cube1.display();
cube1.area();
  cube1.volume();    } }
```

# A Superclass Variable Can Reference a Subclass Object

A major advantage of inheritance is that once you have created a superclass that defines the attributes common to a set of objects, it can be used to create any number of more specific subclasses. Each subclass can precisely tailor its own classification.

```
class RefDemo {
  public static void main(String args[]) {
    BoxWeight weightbox = new BoxWeight(3, 5, 7, 8.37);
    Box plainbox = new Box();
    double vol;

    vol = weightbox.volume();
    System.out.println("Volume of weightbox is " + vol);
    System.out.println("Weight of weightbox is " +
                       weightbox.weight);
    System.out.println();

    // assign BoxWeight reference to Box reference
    plainbox = weightbox;

    vol = plainbox.volume(); // OK, volume() defined in Box
    System.out.println("Volume of plainbox is " + vol);

    /* The following statement is invalid because plainbox
       does not define a weight member. */
//  System.out.println("Weight of plainbox is " + plainbox.weight);
  }
}
```

- when a reference to a subclass object is assigned to a superclass reference variable, you will have access only to those parts of the object defined by the superclass.
- This is why plainbox can’t access weight even when it refers to a BoxWeight object. If you think about it, this makes sense, because the superclass has no knowledge of what a subclass adds to it.

```
  class Animal
   {
       String name="Animal";
void eat()
{
System.out.println("eating...");}
}
```

```
class TestSuper2{
public static void main(String args[]){
Animal d=new Dog();
//d.work();
d.eat();
System.out.println(d.name);
}}
```

```
class Dog extends Animal
{
String name="dog";
void eat()
{
    System.out.println("eating
bread...");
   }

void bark()
{System.out.println("barking...");

}
void work()
{
super.eat();
bark();
```

# When Constructors Are Executed

```
class A
{
    A(int a)
    {
        System.out.println("a constructor");
    }
}
class B extends A
{
    B(int a)
    {
        super(10);
        System.out.println("b constructor");
    }
}
class multi
{
```

# Method Overriding

when a method in a subclass has the same name and type signature as a method in its superclass, then the method in the subclass is said to override the method in the superclass. When an overridden method is called from within its subclass, it will always refer to the version of that method defined by the subclass. The version of the method defined by the superclass will be hidden.

```
// Method overriding.
class A {
  int i, j;
  A(int a, int b) {
    i = a;
    j = b;
  }

  // display i and j
  void show() {
    System.out.println("i and j: " + i + " " + j);
```

```
class B extends A {
  int k;

  B(int a, int b, int c) {
    super(a, b);
    k = c;
  }

  // display k - this overrides show() in A
  void show() {
    System.out.println("k: " + k);
  }
}

class Override {
  public static void main(String args[]) {
    B subOb = new B(1, 2, 3);

    subOb.show(); // this calls show() in B
  }
}
```

When show( ) is invoked on an object of type B, the version of show( ) defined within B is used. That is, the version of show( ) inside B overrides the version declared in A

If you wish to access the superclass version of an overridden method, you can do so by using super. For example, in this version of B, the superclass version of show( ) is invoked within the subclass' version.

```
class B extends A {
  int k;

  B(int a, int b, int c) {
    super(a, b);
    k = c;
  }

  void show() {
    super.show(); // this calls A's show()
    System.out.println("k: " + k);
  }
```

```
// Methods with differing type signatures are overloaded - not
// overridden.
class A {
  int i, j;

  A(int a, int b) {
    i = a;
    j = b;
  }

  // display i and j
  void show() {
    System.out.println("i and j: " + i + " " + j);
  }
}

// Create a subclass by extending class A.
class B extends A {
  int k;

  B(int a, int b, int c) {
    super(a, b);
    k = c;
  }
```

```
  // overload show()
  void show(String msg) {
    System.out.println(msg + k);
  }
}

class Override {
  public static void main(String args[]) {
    B subOb = new B(1, 2, 3);

    subOb.show("This is k: "); // this calls show() in B
    subOb.show(); // this calls show() in A
  }
}
```

# Why Overridden Methods?

overridden methods allow Java to support run-time polymorphism.

**Polymorphism is essential to object-oriented programming for one reason:**

- It allows a general class to specify methods that will be common to all of its derivatives, while allowing subclasses to define the specific implementation of some or all of those methods.
- Overridden methods are another way that Java implements the “one interface, multiple methods” aspect of polymorphism.

# Dynamic Method Dispatch

Dynamic method dispatch is the mechanism by which a call to an overridden method is resolved at run time, rather than compile time.

- Method overriding forms the basis for one of Java's most powerful concepts: *dynamic method dispatch.*
- Dynamic method dispatch is important because this is how Java implements run-time polymorphism.
- Restating an important principle: a superclass reference variable can refer to a subclass object.

- Java determines which version of that method to execute based upon the type of the object being referred to at the time the call occurs. Thus, this determination is made at run time.
- *It is the type of the object being referred to (not the type of the reference variable)* that determines which version of an overridden method will be executed.
- Therefore, if a superclass contains a method that is overridden by a subclass, then when different types of objects are referred to through a superclass reference variable, different versions of the method are executed.

```
// Dynamic Method Dispatch
class A {
void callme() {
System.out.println("Inside A's callme method");
}
}
class B extends A {
// override callme()
void callme() {
System.out.println("Inside B's callme method");
}
}
class C extends A {
// override callme()
void callme() {
System.out.println("Inside C's callme method");
}
}
```

```
class Dispatch {
public static void main(String args[]) {
A a = new A(); // object of type A
B b = new B(); // object of type B
C c = new C(); // object of type C
A  r; // obtain a reference of type A
r = a; // r refers to an A object
r.callme(); // calls A's version of callme
r = b; // r refers to a B object
r.callme(); // calls B's version of callme
r = c; // r refers to a C object
r.callme(); // calls C's version of callme
}
}
```

Note: r being the object reference of A can hold only the subclass object or the object of the same class

# Using Abstract Classes

sometimes we want to create a superclass that determines the nature of the methods that the subclasses must implement then we use abstract classes.

One way this situation can occur is when a superclass is unable to create a meaningful implementation for a method.

Example:

Rules made

- Every institution should work for 8 hours
- Every institution should main the record of students
- Every institution should admit the student if age >6

# Using Abstract Classes

- Certain methods be overridden by subclasses by specifying the **abstract type modifier.**
- **These methods are sometimes referred to as *subclasser responsibility***

because they have no implementation specified in the superclass.

Thus, a subclass must override them—it cannot simply use the version defined in the superclass.

To declare an abstract method, use this general form:

**abstract *type name(parameter-list);*** ***abstract void add(int a,intb);***

As you can see, no method body is present

- Any class that contains one or more abstract methods must also be declared abstract

- Any class that contains one or more abstract methods must also be declared abstract.
- To declare a class abstract, you simply use the **abstract keyword in front of the class keyword** at the beginning of the class declaration.
- There can be no objects of an abstract class.
- That is, an abstract class cannot be directly instantiated with the **new operator.**
- **Such objects would** be useless, because an abstract class is not fully defined.
- Also, you cannot declare abstract constructors, or abstract static methods.
- Any subclass of an abstract class must either implement all of the abstract methods in the superclass, or be itself declared **abstract.**

```
// A Simple demonstration of abstract.
abstract class A {
abstract void callme();
// concrete methods are still allowed in abstract classes
void callmetoo() {
System.out.println("This is a concrete method.");
}
}
class B extends A {
void callme() {
System.out.println("B's implementation of callme.");
}
}
```

```
class AbstractDemo {
public static void main(String args[]) {
 B b= new B();
b.callme();
b.callmetoo();
}
}
```

# Using final with Inheritance

- The keyword **final has three uses.**
- **First, it can be used to create the equivalent of a named** constant.
- The other two uses of **final apply** to inheritance.

**1)Using final to Prevent Overriding**

- While method overriding is one of Java's most powerful features, there will be times when
- you will want to prevent it from occurring. To disallow a method from being overridden,
- specify **final as a modifier at the start of its declaration. Methods declared as final cannot**
- be overridden.

The following fragment illustrates **final:**

```
final class A {
final void meth() {
System.out.println("This is a final method.");
}
}
class B extends A {
void meth() { // ERROR! Can't override.
System.out.println("Illegal!");
}
}
```

**2)Using final to Prevent Inheritance**

Here is an example of a **final class:**

```
final class A {
// ...
}
// The following class is illegal.
class B extends A { // ERROR! Can't subclass A
// ...
}
```

As the comments imply, it is illegal for **B to inherit A since A is declared as final**